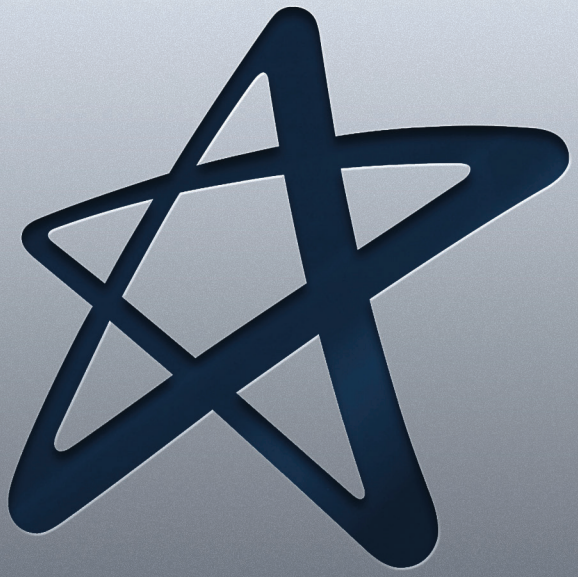
Modelagem de Negócios



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Utilizando UML para Documentar Regras de Negócios

Responsável pelo Conteúdo:
Prof. Me. Luiz Carlos Reis

Revisão Textual:
Prof. Me. Claudio Brites

UNIDADE Utilizando UML para Documentar Regras de Negócios

- Introdução;
- Diagramas Estruturais;
- Diagramas Comportamentais.



OBJETIVOS DE APRENDIZADO

- No processo de desenvolvimento de *software* sempre foi um grande desafio atender às necessidades dos usuários finais com maior rapidez e precisão, devido à complexidade disso em função de grandes demandas por sistemas que exigem cada vez mais integrações com outros sistemas;
- Nesse raciocínio, podemos ter a *UML* (*Unified Modeling Language*) como aliada, a partir da qual podemos ter diagramas **estruturais** e diagramas **comportamentais** que podem nos ajudar a criar uma notação simples e objetiva que facilitará o entendimento pela equipe de sistemas.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Existem algumas notações para modelagem de negócios, mas daremos mais ênfase a duas delas, que são:

- ***UML – Unified Modeling Language:*** notação voltada para processos de *software* e que facilita muito o entendimento entre a equipe de negócios e a equipe de sistemas. Essa notação abordaremos nesta unidade;
- ***BPMN – Business Process Modeling Notation*:** é uma notação mais moderna que permite modelar processos e que facilita a comunicação entre áreas de negócio. Possui também um mapeamento de elementos para automatizar os processos – essa notação será abordada nas próximas unidades.

No processo de desenvolvimento de *software* sempre foi um grande desafio atender às necessidades dos usuários finais com maior rapidez e precisão, devido à complexidade disso em função de grandes demandas por sistemas que exigem cada vez mais integrações com outros sistemas.

Para essa integração, podemos ter a *UML* (*Unified Modeling Language*) como aliada, a partir da qual podemos ter diagramas **estruturais** e diagramas **comportamentais** que podem nos ajudar a criar uma notação simples e objetiva que facilitará o entendimento pela equipe de sistema.

Primeiramente, entenderemos como surgiu a UML. Ela se originou da necessidade de produzir sistemas orientados a objetos (OO), visto que as linguagens com essa estrutura e todas as técnicas até o momento eram voltadas para a análise estruturada.

Para atender a essa necessidade, foram criadas algumas técnicas:

- **Grady Booch:** técnica Booch;
- **Ivar Jacobson:** técnica OOSE;
- **James Rumbaugh:** técnica OMT.

Em meados de 1994, Booch, Rumbaugh e Jacobson solidificaram forças para combinar suas metodologias (Booch, OMT e OOSE). Por volta de 1996, surgiu então a UML (Linguagem de Modelagem Unificada), com o propósito de juntar o que havia de melhor nas três metodologias.

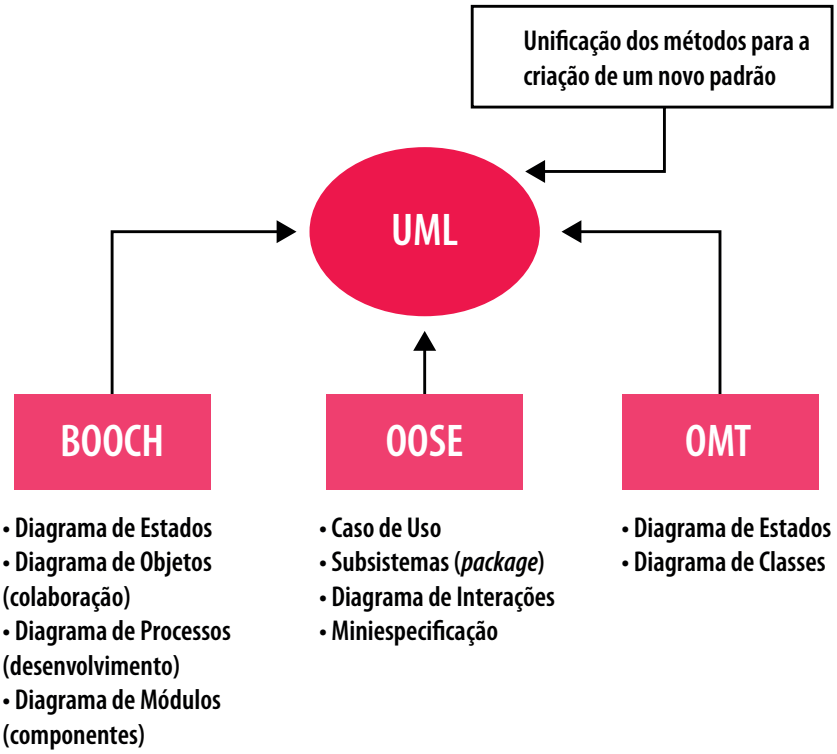


Figura 1

A UML veio com a finalidade de padronizar a modelagem orientada a objetos e demonstrar de forma gráfica e fácil a interação entre as equipes de desenvolvimento. Antes dela, existiram outras metodologias de modelagem orientada a objetos, mas essas causavam sempre conflitos entre as comunidades de desenvolvedores orientados a objetos.

A UML tem como principais objetivos:

- Documentar a modelagem de sistemas utilizando os conceitos da orientação a objetos;
- Tornar os executáveis e os métodos conceituais;
- Padronizar uma linguagem de modelagem padrão entre o homem e a máquina.

Todos os sistemas possuem comportamentos dinâmicos e estruturas estáticas e a UML também documenta esses modelos por meio dos seguintes diagramas:

Diagramas Estruturais (estático): Devem ser utilizados para especificar detalhes da estrutura do sistema (parte estática), por exemplo: classes; métodos; *interfaces*; *namespaces*; serviços; como componentes devem ser instalados; como deve ser a arquitetura do sistema etc. São exemplos de diagramas estruturais:

- Diagrama de classe;
- Diagrama de objeto;
- Diagrama de componentes.

Diagramas Comportamentais (dinâmico): Devem ser utilizados para especificar detalhes do comportamento do sistema (parte dinâmica), por exemplo: como as funcionalidades devem funcionar; como um processo de negócio deve ser tratado pelo sistema; como componentes estruturais trocam mensagens; e como respondem às chamadas etc. São exemplos de diagramas comportamentais:

- Diagrama caso de uso;
- Diagrama de sequência;
- Diagrama de colaboração;
- Diagrama de estados;
- Diagrama de atividades.

Diagramas Estruturais

Diagrama de Classes

Sua finalidade é demonstrar a estrutura estática das classes de um sistema. Essas classes podem estar relacionadas através de diversas maneiras: associação (conectadas entre si); dependência (uma classe depende ou usa outra classe); especialização (uma classe é uma especialização de outra classe); ou em pacotes (classes agrupadas por características similares).

Independentemente dos relacionamentos, os mesmos deverão ser exibidos no diagrama de classes juntamente com as suas estruturas internas, que são seus atributos e operações. Esse é considerado estático, visto que a estrutura descrita é sempre válida em qualquer ponto do ciclo de vida do sistema.

Uma classe representada por meio desse diagrama pode ser implementada utilizando-se qualquer linguagem de programação orientada a objetos. Para se criar um diagrama de classes, essas devem estar identificadas, descritas e relacionadas entre si.

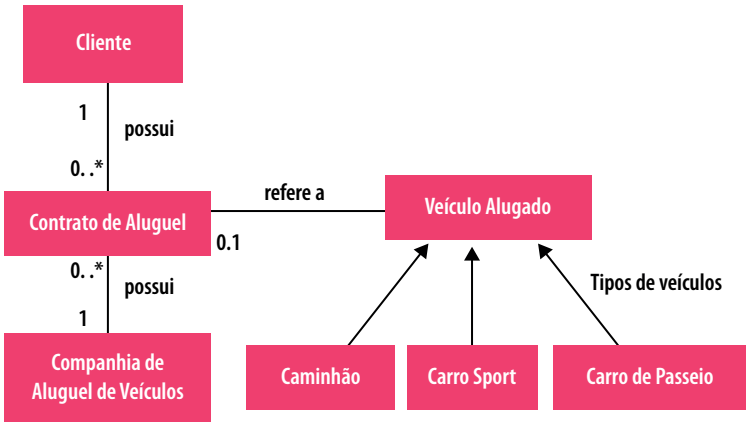


Figura 2

Diagrama de Objetos

É uma variação do diagrama de classes, visto que o mesmo utiliza quase sempre a mesma notação. A diferença entre eles é que o diagrama de objetos evidencia os objetos que foram instanciados das classes, ou seja, os objetos que foram criados a partir do diagrama de classes.

O diagrama de objetos demonstra, como se fosse uma foto em certo momento de sua execução. Não tem a mesma importância que os diagramas de classes, mas são muito úteis para exemplificar cada ocorrência da classe, o que facilita muito sua compreensão.

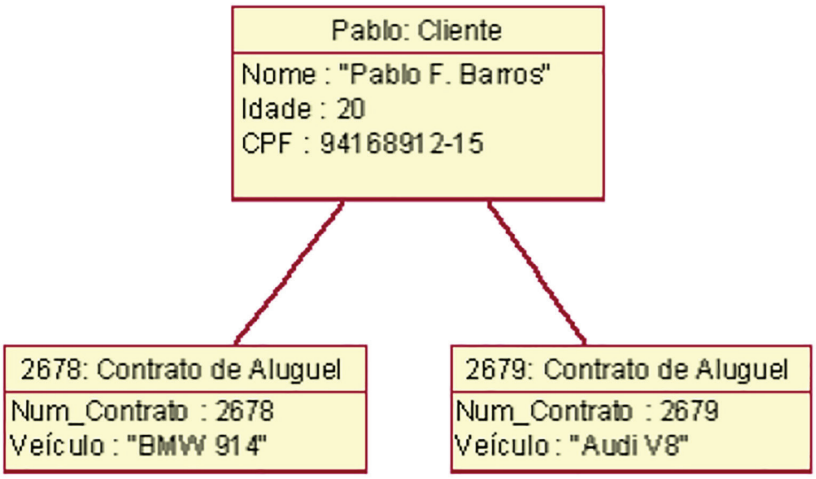


Figura 3
Fonte: Acervo do conteudista

Diagrama de Componentes

Esse diagrama demonstra o sistema por um lado funcional, expondo as relações entre seus componentes e a organização de seus módulos durante sua execução, visto que descreve os componentes de *software* e suas dependências.

Esses componentes são a implementação na arquitetura física do conceito e da funcionalidade, definidos na arquitetura lógica, como, por exemplo, as classes, objetos e seus relacionamentos.

Para representar um componente em UML, é utilizado um retângulo com uma elipse e dois retângulos menores do seu lado esquerdo, com o nome do componente escrito abaixo ou dentro do seu símbolo.

A linha tracejada com uma seta é utilizada para demonstrar a dependência entre componentes, simbolizando que um componente necessita do outro para possuir uma definição completa.

Por meio do diagrama de componentes, fica mais visível detectar que arquivos .dll são necessários para executar a aplicação por exemplo.

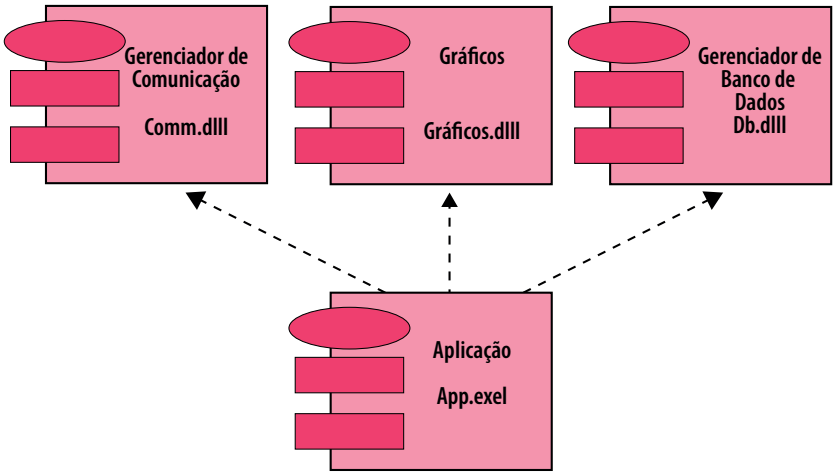


Figura 4

Os componentes podem definir interfaces que são visíveis para outros componentes e essas podem ser tanto definidas no nível de codificação, quanto em interfaces binárias usadas em tempo real.

Uma interface é demonstrada como uma linha partindo do componente e com um círculo na outra extremidade; e o nome, colocado junto do círculo no final da linha. Dependências entre componentes podem então apontar para a interface do componente que está sendo usada.

Diagramas Comportamentais

Diagrama de Caso de Uso

A finalidade desse diagrama é descrever os requisitos funcionais (define o que o sistema fará) do sistema. O objetivo é mostrar que desenvolvedores e clientes estão de acordo sobre o que será desenvolvido.

Requisitos funcionais estão relacionados aos serviços que o sistema deve fornecer e especificam também o que o sistema deve fazer.

Exemplos:

- O sistema deve permitir cancelar um pedido;
- O sistema deve realizar uma venda;
- O sistema deve permitir estornar o pagamento.

Esse diagrama deve fornecer uma visão clara sobre o que o sistema deve fazer para que posteriormente a execução dos testes possa verificar se o sistema trabalha adequadamente.

O diagrama de caso de uso representam as interações dos atores com o sistema. Atores são todas as entidades que passam a interagir com o sistema, ou seja, representa um papel que um ser humano, um dispositivo de *hardware* ou até outro sistema desempenha com o sistema.

Atores podem ser uma das seguintes entidades:

- Pessoa;
- Outro sistema informático;
- Equipamento *hardware* especializado;
- Passagem de tempo.

É sempre o ator que causa o estímulo e sempre estará fora do sistema. São representados pela Figura 5.

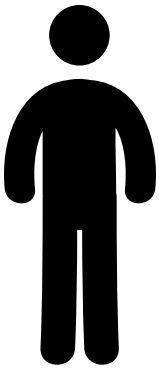


Figura 5

Um caso de uso captura uma funcionalidade do sistema, mas não revela a estrutura e o comportamento interno do sistema. Existem várias formas para descrever casos de uso:

- **Um texto contínuo:**
  - » O cliente chega à livraria. No terminal de consulta, o sistema mostra as formas de pesquisa (por título da obra, pelo nome do autor, pelo nome da editora).
- **Uma sequência numerada:**
  - » Cliente chega à livraria e dirige-se a um terminal de consulta;
  - » O sistema exibe as formas de pesquisa (por título da obra, pelo nome do autor, pelo nome da editora);
  - » O cliente escolhe a forma de pesquisa que lhe interessa;
  - » O sistema exibe as informações sobre o produto desejado;

- **Utilização de tabelas:**

Tabela 1

Cliente	Sistema
1. Cliente chega à livraria e dirige-se a um terminal de consulta.	2. Sistema exibe as formas de pesquisa (por título da obra, pelo nome do autor, pelo nome da editora).
3. Cliente escolhe a forma de pequisa que lhe interessa.	4. Sistema exibe as informações sobre o produto desejado.

A finalidade do diagrama de caso de uso é descrever e definir os requisitos funcionais de um sistema.

A comunicação com o sistema é realizada pelos atores e representada pelo caso de uso, com a finalidade de demonstrar a sequência de ações que o sistema deverá executar.

Por meio dele, será possível visualizar um alto nível de abstração, quais elementos (atores) interagirão com o sistema em cada funcionalidade (Figura 6).

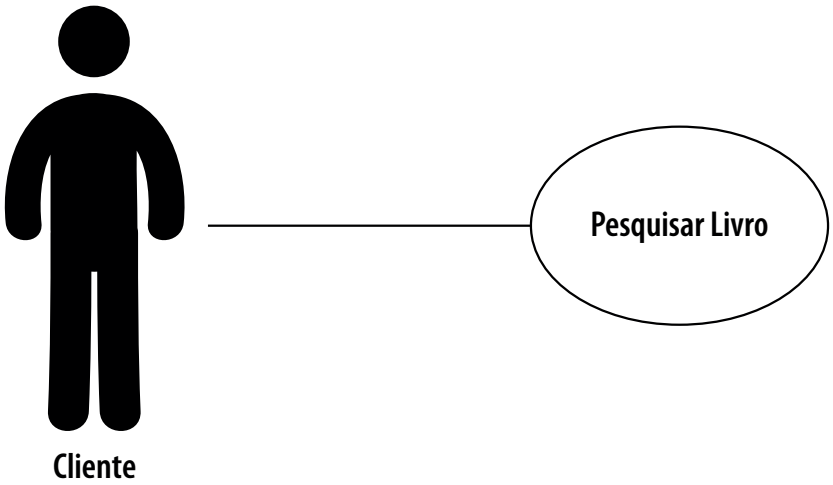


Figura 6

As ações devem ser únicas e devem estar na perspectiva do ator que dispara o caso de uso. Deve-se também ser iniciado com verbo no infinitivo (Figura 7).



Figura 7

Relações em Casos de Uso:

- **<<include>>:** Essa relação incorpora o comportamento de uma ação do caso de uso a outra ação do caso de uso e indica obrigatoriedade.

No exemplo abaixo, para efetuar o pedido como para cancelar o pedido, o cliente deve primeiramente estar logado no sistema.

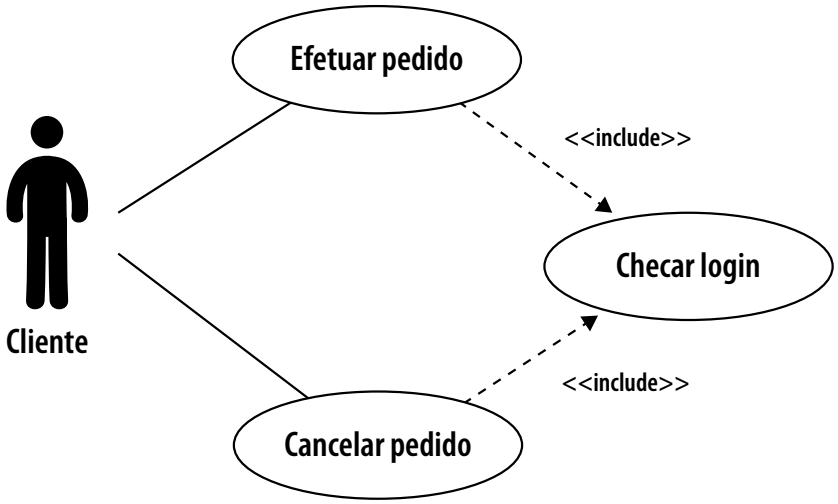


Figura 8

- **<<extend>>:** Essa relação indica que o comportamento estendido poderá ou não ser usado, ou seja, é opcional. Notamos no exemplo a seguir que, para efetuar o pedido, o mesmo poderá pagar à vista ou com boleto.

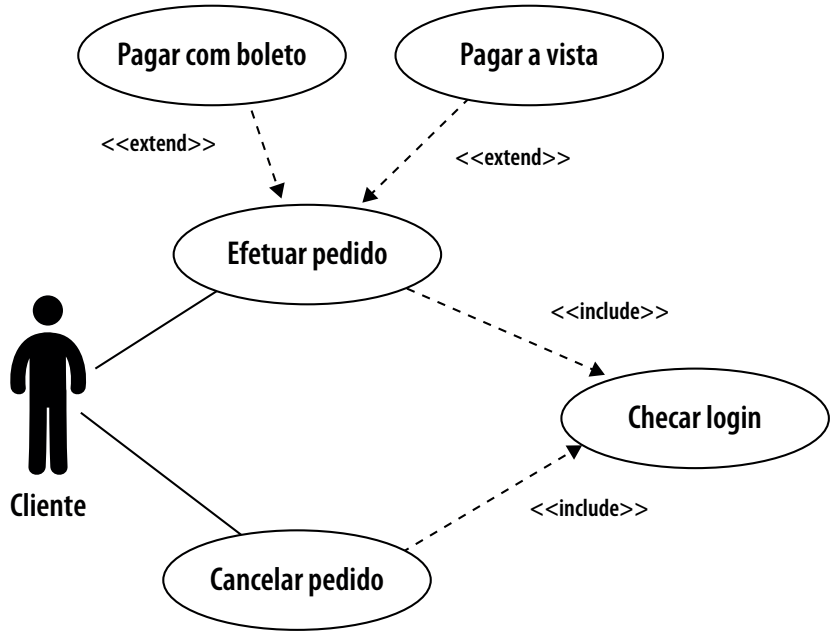


Figura 9

- **Generalização**: Essa relação é utilizada para criar uma ação específica baseada em uma ação de uso geral. Nesse caso, podemos ter vários segmentos do item no qual ele generalizou.

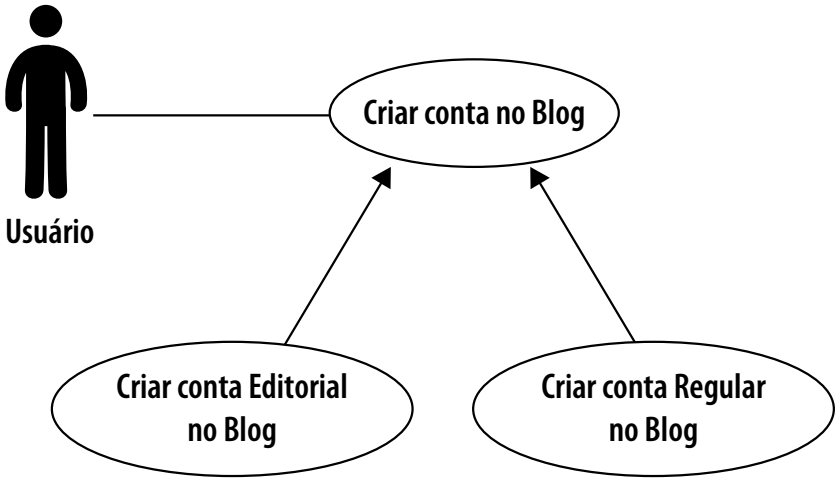


Figura 10

Uma boa prática para detalhar cada caso de uso é ter um modelo que descreva a interação entre o ator e o sistema, conforme Tabela 2.

Tabela 2

Caso de Uso: Encomendar Livro	
Atores: Vendedor, Cliente	
Descrição: Vendedor informa o título do livro desejado pelo Cliente. Sistema solicita dados do Cliente. Sistema gera pedido de encomenda do livro.	
Sequência Típica de Evento	
Atores	**Sistema**
1. Cliente informa ao Vendedor o título do livro desejado.	
2. Vendedor informa título do livro ao Sistema.	3. Solicita dados do Cliente.
4. Informa dados do Cliente.	5. Abre pedido de encomenda do livro.
	6. Informa a data de previsão de chegada do livro.
	7. Encerra operação.
Sequência Alternativa de Eventos	
Não há.	

Vamos a um exemplo prático: **SISTEMA DE LIGAÇÕES TELEFÔNICAS.**

Desenvolver uma aplicação para controlar as ligações telefônicas, a fim de analisar se o valor pago mensalmente está correto.

A aplicação deverá exibir as ligações efetuadas em um determinado período e contabilizar o valor a ser pago.

Para cada ligação, a aplicação deverá registrar: a data e hora, quantidade de minutos gastos, a quantidade de pulsos e o telefone discado.

A aplicação deverá controlar a agenda telefônica, armazenando o número do telefone e o nome da pessoa de contato. O usuário poderá escolher um dos registros da agenda ou discar diretamente o número do telefone.

Critérios para o cálculo dos pulsos:

- A ligação ao ser completada já conta um pulso e a cada 4 minutos de conversação, adiciona mais 1 pulso;
- O valor para cada pulso custa R$ 0.45 para ligações locais;
- Cada ligação no fim de semana contabiliza somente 1 pulso, independente do número de minutos de conversação.

Podemos representar as regras de negócios acima através do diagrama da Figura 11.

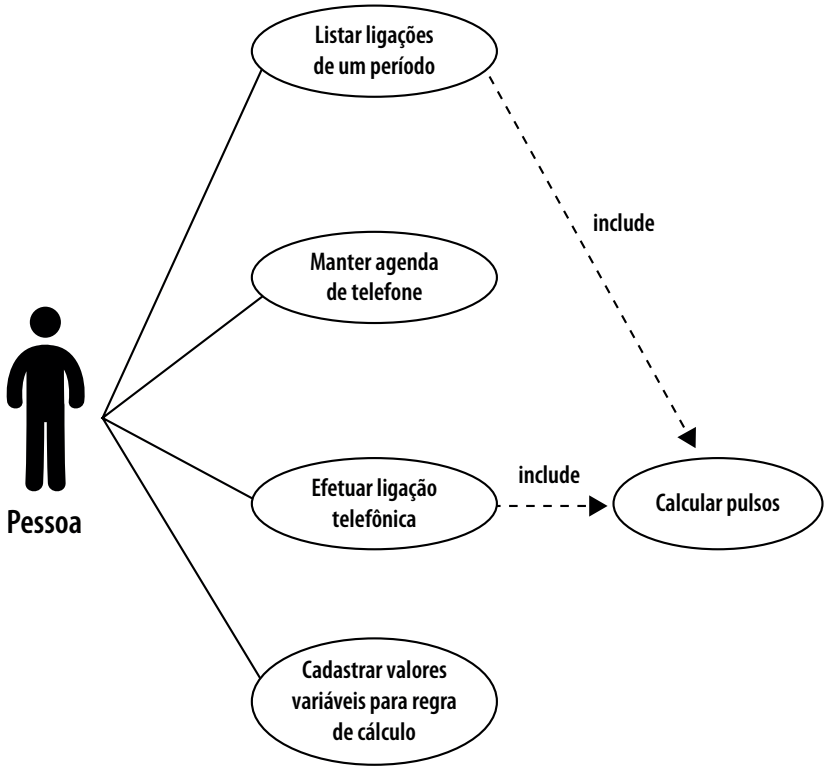


Figura 11

Diagrama de Estado

Esse diagrama é um complemento do diagrama de classes. Mostra todos os estados possíveis no qual um objeto de sua respectiva classe pode se encontrar. Mostra também quais são os eventos dos sistemas que provocam tais mudanças.

Na Figura 12, podemos ver um pedido realizado pela internet. Quando você acompanhar o status desse pedido, será demonstrado qual o estado do mesmo.

Esse diagrama não é utilizado para todas as classes de um sistema, mas apenas para aquelas que possuem um número definido de estados conhecidos e em que o comportamento das classes é afetado e modificado pelos diferentes estados.

Tais diagramas capturam o ciclo de vida dos objetos, subsistemas e sistemas e mostram os estados que um objeto pode possuir e como os eventos afetam esses estados ao passar do tempo.

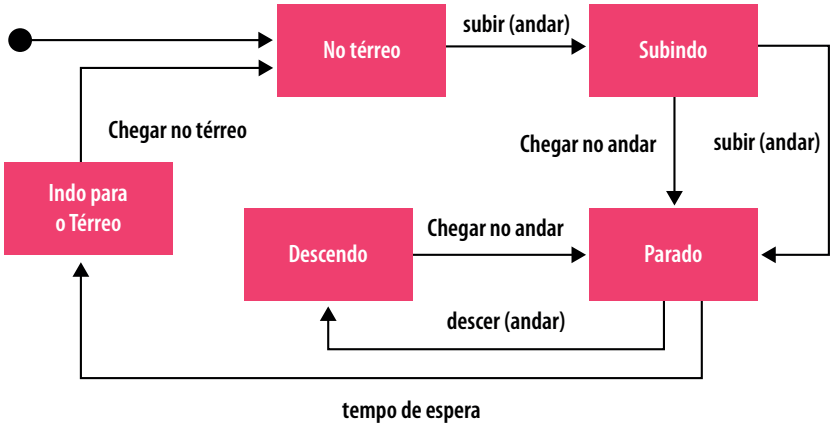


Figura 12

Esse diagrama possui um ponto de início e pode ter vários pontos de finalização. O ponto de início é o estado inicial e é demonstrado como um círculo todo preenchido. O ponto de finalização é o estado final e é demonstrado como um círculo em volta de outro círculo menor preenchido.

O estado é representado através de um retângulo com cantos arredondados. Entre os estados estão as transições, mostradas como uma linha com uma seta no final de um dos estados.

A transição pode ser nomeada com seu evento causador, ou seja, quando o evento acontece e a transição de um estado para outro é executada ou disparada.

Em uma transição de estado, normalmente existirá um evento ligado a ela. Caso um evento seja anexado a uma transição, essa será executada quando o evento ocorrer.

Diagrama de Sequência

Esse diagrama serve para demonstrar a colaboração dinâmica entre os vários objetos de um sistema. Deverá exibir a sequência de ações entre os objetos e demonstrar a interação entre eles. Consiste em exibir em linhas verticais o número de objetos que fazem parte da sequência de uma funcionalidade. As mensagens enviadas para cada objeto são demonstradas através de setas entre os objetos, as quais se relacionam entre si.

Os diagramas de sequência possuem dois eixos:

- **Vertical**: tem a finalidade de exibir o tempo;
- **Horizontal**: tem a finalidade de exibir os objetos envolvidos na sequência de certa atividade, também permitindo exibir as interações para um cenário específico de certa atividade do sistema.

Através do eixo horizontal, serão exibidos os objetos envolvidos na sequência, o qual deve ser representado por um retângulo. O eixo vertical deve ser pontilhada e representar a linha de vida do objeto, indicando a execução desse durante a sequência. Como exemplo citamos: mensagens recebidas ou enviadas e ativação de objetos.

Para representar a comunicação entre os objetos, será utilizada uma linha com setas horizontais simbolizando as mensagens entre as linhas de vida dos objetos. Essas mensagens podem possuir números sequenciais e são utilizados para tornar mais explícita as sequências no diagrama.

A seguir, um exemplo do diagrama de sequência demonstrando as interações dos objetos em um restaurante.

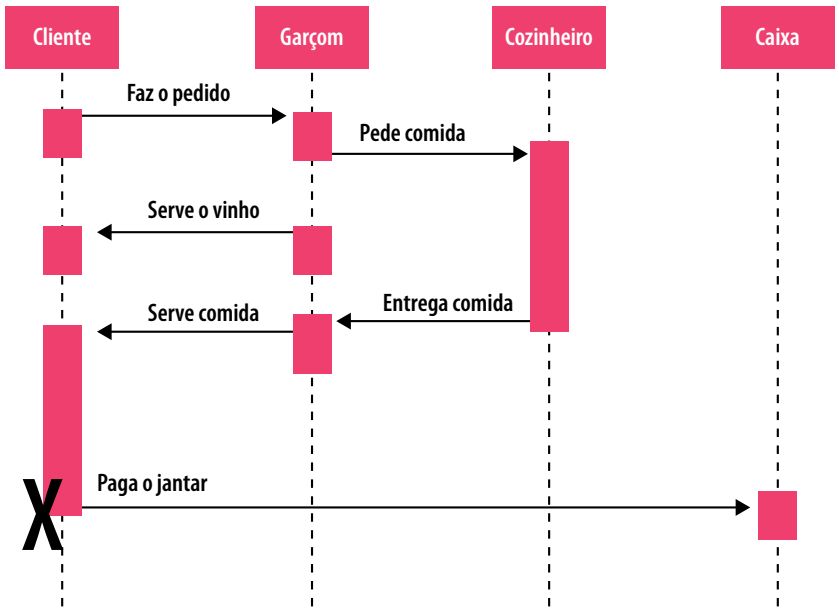


Figura 13

Diagrama de Colaboração

Esse diagrama deve demonstrar a colaboração dinâmica entre os objetos. Normalmente pode-se escolher entre utilizar o diagrama de colaboração ou o diagrama de sequência, mas opte por esse caso precise identificar os objetos com os seus relacionamentos.

Caso a ênfase do diagrama seja o decorrer do tempo, então será melhor escolher o diagrama de sequência. Mas se a ênfase for o contexto do sistema, é melhor dar prioridade ao diagrama de colaboração.

Esse diagrama é desenhado parecido como um diagrama de objeto, mas esse demonstra seus relacionamentos. As setas de mensagens são desenhadas entre os objetos para exibir o fluxo de mensagens entre eles. As mensagens são nomeadas e, entre outras coisas, mostram a ordem em que as mensagens são enviadas.

O mesmo também permite exibir as condições, interações, valores de resposta, também podendo conter objetos ativos que executam paralelamente com outros.

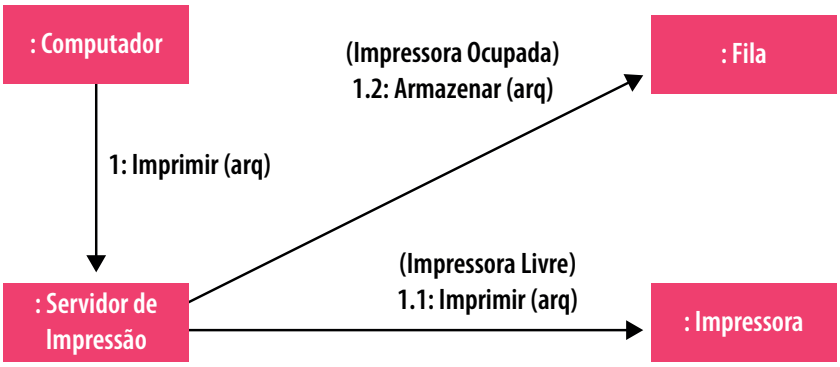


Figura 14

Diagrama de Atividade

A finalidade desse diagrama é capturar as ações e seus resultados. Focam no trabalho executado e na implementação de uma operação (método), e suas atividades em uma instância de um objeto.

Esse diagrama é uma variação do diagrama de estado, mas com outro ponto de vista. Seu objetivo é capturar as ações e seus resultados em termo das mudanças de estados dos objetos.

No diagrama de atividade, os estados mudam para um próximo estágio quando uma ação é executada.

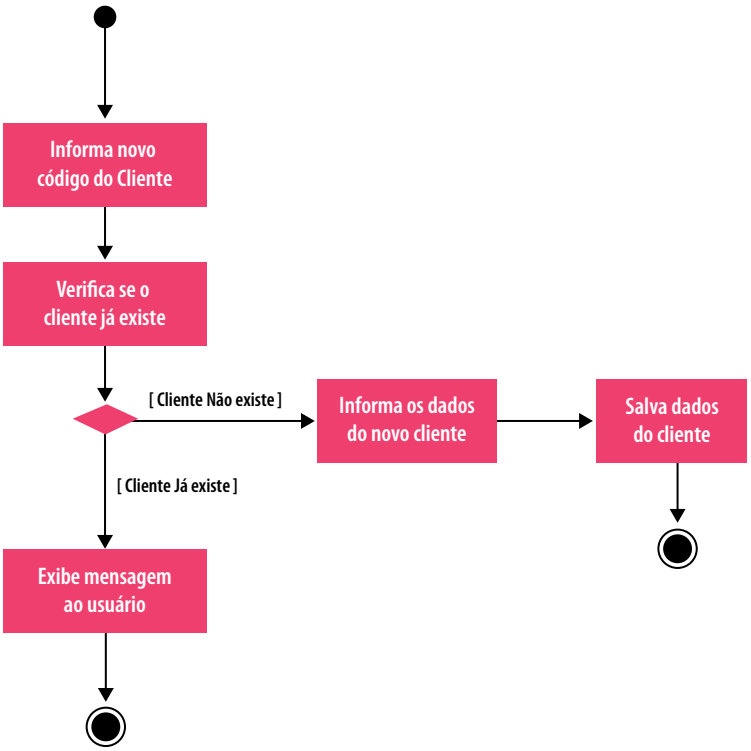


Figura 15

Outra diferença entre o diagrama de atividade e os de estado é que esses podem ser colocados como *swimlanes* (raias), ou seja, agrupar atividades que permitem identificar quem é responsável e onde essas atividades residem na organização. É representada por retângulos que englobam todos os objetos que estão ligados a ela.

Por meio desse diagrama, teremos uma maneira alternativa de demonstrar quais interações ocorrem, tendo a possibilidade também de expressar como as ações são executadas, o que elas fazem, quando elas são executadas e onde elas acontecem.

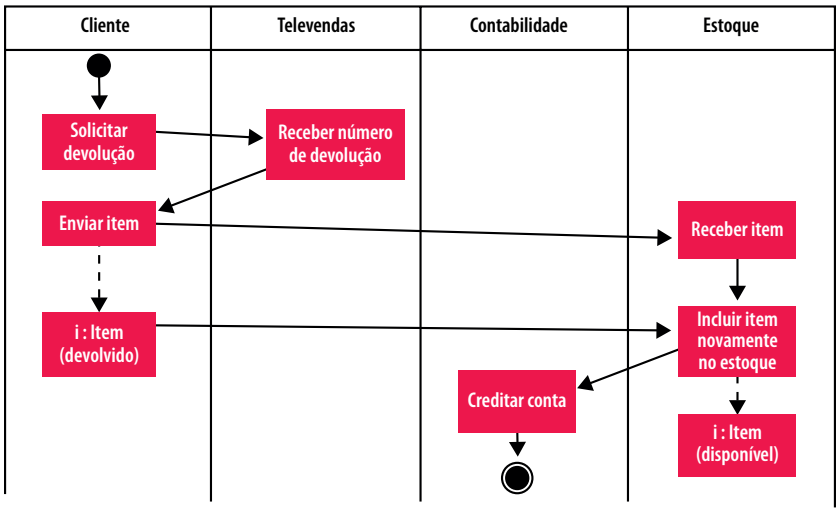


Figura 16

O diagrama de atividade pode ser usado com diferentes propósitos:

- Capturar os trabalhos que serão executados quando uma operação é disparada;
- Capturar o trabalho interno em um objeto;
- Demonstrar como um grupo de ações relacionadas pode ser executado, e como elas vão afetar os objetos em torno delas;
- Demonstrar como uma instância pode ser executada em termos de ações e objetos;
- Demonstrar como um determinado negócio funciona em termos de trabalhadores, fluxos de trabalho, organização e objetos.

É por meio desse diagrama que se permite demonstrar o fluxo sequencial das atividades executadas por uma operação específica do sistema. Engloba também os estados de ação no qual há a especificação de uma atividade a ser desempenhada por uma operação do sistema.

Permite-se também identificar as decisões e condições demonstradas no losango abaixo. As execuções paralelas também podem ser demonstradas na barra vermelha horizontal, indicando assim a atividade em paralelo.



Figura 17
Fonte: Acervo do conteudista

Em Síntese

Elaborar técnicas de análise e definir padrões sempre foi uma tarefa árdua, e muitas organizações não compreendem que a modelagem de negócios é essencial para tomada de decisão, visto que os projetos variam muito e as análises diferem em perspectiva, quantidade e nível de detalhe.

Elaborar um padrão de notação particular ou até ter uma modelagem pode parecer uma boa maneira de introduzir consistência na organização. As organizações, aliás, deveriam empregar cada vez mais excelentes profissionais de modelagem de negócios que tenham domínio de notações de modelagem diferentes e também da arte de trabalhar com as partes interessadas para dominar assim a arte da comunicação em todos os níveis da organização

Por meio dessas ações, as organizações tendem a receber muito bem as ações de modelagem de processos de negócio e seus impactos na organização passam a ser os mais benéficos possíveis. Isso dá a visibilidade necessária para as etapas futuras à modelagem, onde serão almejadas as mudanças consistentes de curto e longo prazo, pois essas farão a organização surgir no mercado com mais força, com processos corporativos bem padronizados, almejando ganho expressivo de produtividade e eficiência, além de permitir medições, análises e aperfeiçoamento da gestão estratégica.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

Aula 06 – Visão Geral de UML
https://youtu.be/X1tgd97_i2A

UML – Caso de Uso – Introdução
https://youtu.be/fx0mBsgS4Uw

Introdução a UML e Diagrama de classes – Aula 05
https://youtu.be/BQiZnEw4Q10

Leitura

UML Fundamentos
https://bit.ly/2ZG4QIg

Referências

BROCKE, J. V.; ROSEMANN, M. **Manual de BPM**: gestão de processos de negócio. Porto Alegre: Bookman, 2013. *e-book*

ENOKI, C. H. **Gestão de processos de negócio:** uma contribuição para a avaliação de soluções de *Business Process Management* (*BPM*) sob a ótica da estratégia de operações. Dissertação (Mestrado) – Universidade de São Paulo, São Paulo. 2006.

FOWLER, M. **UML Essencial**. 3. ed. Porto Alegre: Bookman, 2005.

JACOBSON, I.; BOOCH, G.; RUMBAUGH, J. ***The Unified Software Development Process***. Boston: Addison Wesley, 1999.

LAUDON, K. C.; LAUDON, J. P. **Sistemas de informação gerenciais**. 9. ed. Sao Paulo: Pearson Prentice Hall, 2012. Disponível em: <http://www.teses.usp.br/teses/disponiveis/3/3136/tde-01122006-170526/pt-br.php>. Acesso em: 28/01/2019.


Cruzeiro do Sul
Educacional